



Our Full-Stack JavaScript Developer Roadmap

Mission: Take you from beginner software developer → confident JavaScript engineer → React frontend developer → Node.js backend developer → full-stack engineer capable of designing, building, testing, debugging and deploying real applications.

Target: ~8–14+ months at 10–15 hours/week.

Philosophy: Understanding > memorization. Building > watching. Debugging > copying.

0. The Rules We Are Going To Follow

These are more important than the individual technologies.

Rule 1 — We don't learn frameworks to avoid learning fundamentals

You will understand:

```
JavaScript
  ↓
Browser
  ↓
HTTP
  ↓
Node
  ↓
Database
  ↓
React
```

before relying heavily on abstractions.

Rule 2 — Every phase has a project

You don't "finish JavaScript."

You demonstrate that you understand JavaScript by **building something with it**.

Rule 3 — No tutorial hell

Our workflow will be:

```
Learn concept
  ↓
Follow example
  ↓
Close tutorial
  ↓
Build from memory
  ↓
Get stuck
  ↓
Debug
  ↓
Research
  ↓
Finish
  ↓
Explain what you built
```

Getting stuck is part of the curriculum.

Rule 4 — AI is a teacher, not your autopilot

You can absolutely use me.

But our workflow will **not** be:

```
"Here's my error. Fix it."
```

Instead:

```
"Here's what I think is happening. Here's what I tried. Here's the error. Help me understand it."
```

I'll deliberately make you think when you're learning something important.

Rule 5 — Git from Day 1

Every meaningful project:

```
Git
  ↓
GitHub
  ↓
README
  ↓
commits
```

Eventually:

```
branch
  ↓
feature
  ↓
pull request
  ↓
merge
```

Phase 0 — Developer Environment

Duration

Days 1–3

Learn

- Linux terminal
- VS Code
- Node.js
- npm
- Git
- GitHub
- Browser DevTools

Basic commands:

```
pwd
ls
cd
mkdir
touch
cp
mv
rm
cat
grep
find
```

Git:

```
git init
git status
git add
git commit
git log
git branch
git switch
git merge
git push
git pull
```

Deliverable

Create:

```
javascript-learning/
├── README.md
└── hello.js
```

Run it with Node.

Push it to GitHub.

Milestone

You understand:

"What happens between me writing a file and my computer executing it?"

Phase 1 — Web Fundamentals

Weeks 1–3

Before going deep into JavaScript, understand the environment JavaScript lives in.

HTML

Learn:

- Semantic HTML
- Forms
- Inputs
- Links
- Images
- Tables
- Accessibility
- Document structure

CSS

Learn:

- Selectors
- Cascade
- Specificity
- Box model
- Display
- Positioning
- Flexbox
- Grid
- Responsive design
- Media queries
- Transitions

Projects

Build:

Project 1

Personal profile page

Then:

Project 2

Responsive landing page

No React.

No Tailwind.

No component library.

Pure:

```
HTML
CSS
```

Why?

Because when you eventually write:

```
<div className="flex ...">
```

you should understand what the browser is actually doing.

Phase 2 — JavaScript Core

Weeks 3–8+

This is our **most important phase**.

The senior developer was absolutely right to emphasize the JavaScript engine.

2.1 Language Fundamentals

Master:

```
Variables
Constants
Data types
Operators
Expressions
Conditionals
Loops
Functions
Return values
Parameters
Arguments
```

2.2 Data Structures

Learn deeply:

```
Arrays
Objects
Maps
Sets
```

Then:

```
Stack
Queue
```

You don't need to become a computer-science academic yet.

But you should understand why different data structures exist.

2.3 Functional Array Methods

Master:

```
map()
filter()
reduce()
find()
findIndex()
some()
every()
sort()
```

You shouldn't just know how to call them.

You should understand:

What data comes in?

What data comes out?

Does the original array change?

2.4 Scope & Execution

Now we start getting serious.

Learn:

- Global scope
- Function scope
- Block scope
- Lexical scope
- Closures
- Execution context
- Call stack

Understand:

```
function outer() {  
  let count = 0;  
  
  return function inner() {  
    count++;  
    return count;  
  };  
}
```

Why does `inner()` remember `count` ?

Closure.

2.5 `this`

Understand:

`this`

in:

- Global context
- Object methods
- Regular functions
- Arrow functions
- Constructors
- Classes
- `call()`
- `apply()`
- `bind()`

Don't memorize rules blindly.

Learn why they behave differently.

2.6 Objects & Prototypes

Learn:

Object references
Mutation
Immutability
Prototype chain
prototype
__proto__
Object.create()
Classes
Inheritance

You don't need to become a prototype wizard.

But when you encounter:

```
user.toString()
```

you should understand roughly **where that method comes from**.

2.7 Modules

Learn:

```
export  
import
```

Understand:

```
ES Modules  
CommonJS
```

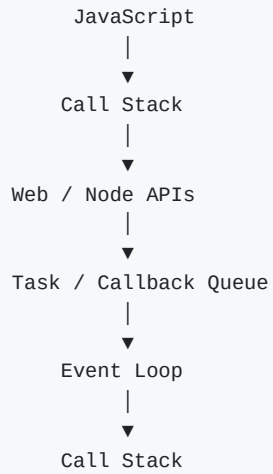
We'll use modern ES modules primarily.

Phase 3 — JavaScript Runtime & Asynchronous Programming

Weeks 6–9

This is where we incorporate the senior developer's strongest advice.

You need a mental model like:



And additionally:

```
Microtask Queue
  ↑
Promises
  ↑
async / await
```

Learn

Synchronous execution
Callbacks
Promises
async/await
Promise chaining
Promise.all()
Promise.race()
Promise.allSettled()
try/catch

Then understand the **event loop**.

You should be able to predict:

```
console.log("A");

setTimeout(() => console.log("B"), 0);

Promise.resolve().then(() => console.log("C"));

console.log("D");
```

without running it.

Phase 4 — Browser & DOM

Weeks 7–10

Now connect JavaScript to the browser.

Learn:

- DOM
- Events
- Event bubbling
- Event capturing
- Event delegation
- Forms
- Validation
- localStorage
- sessionStorage
- Browser APIs
- Fetch
- JSON

Learn browser DevTools:

- Console
- Elements
- Network
- Sources
- Application
- Performance

Project

Vanilla JavaScript Task Manager

Features:

- Create task
- Edit task
- Delete task
- Complete task
- Search
- Filter
- Sort
- Persist to localStorage

No React.

No backend.

No libraries.

Phase 5 — API & HTTP Fundamentals

Weeks 9–11

Before Node/Express, understand the web itself.

Learn:

- HTTP
- HTTPS
- DNS
- URLs
- Requests
- Responses
- Headers
- Bodies
- Cookies
- JSON

HTTP methods:

- GET
- POST
- PUT
- PATCH
- DELETE

Status codes:

- 2xx
- 3xx
- 4xx
- 5xx

Especially:

- 200
- 201
- 204
- 400
- 401
- 403
- 404
- 409
- 422
- 500

REST Architecture

Understand:

```
Resource
Endpoint
HTTP method
Request
Response
Status code
Representation
```

Example:

```
GET    /api/users
GET    /api/users/42
POST   /api/users
PATCH /api/users/42
DELETE /api/users/42
```

Understand why this is different from:

```
/getUser
/createUser
/deleteUser
```

Phase 6 — Node.js

Weeks 11–14

Now JavaScript moves outside the browser.

Learn:

```
Node runtime
npm
package.json
modules
ESM
CommonJS
process
environment variables
fs
path
events
streams
HTTP
```

Build small Node programs.

For example:

```
CLI calculator
CLI todo
File organizer
JSON data processor
Simple HTTP server
```

Important concept

Understand:

```
Browser JavaScript
vs
Node JavaScript
```

The language is largely the same.

The **runtime environment** is different.

Phase 7 — Databases & SQL

Weeks 13–17

This is where I'm deliberately modifying the two roadmaps.

SQL first.

Start with:

PostgreSQL

Learn:

```
CREATE TABLE
INSERT
SELECT
UPDATE
DELETE
```

Then:

```
WHERE
ORDER BY
GROUP BY
HAVING
LIMIT
OFFSET
```

Then:

- Primary keys
- Foreign keys
- Constraints
- Relationships
- Indexes
- Transactions
- Joins
- Normalization

Especially:

- INNER JOIN
- LEFT JOIN

Understand:

```
graph TD
  users --> posts
  users --> comments
```

before hiding everything behind an ORM.

Phase 8 — Express & REST API Development

Weeks 16–20

Now Node + HTTP + databases come together.

Learn:

- Express
- Routing
- Middleware
- Controllers
- Services
- Validation
- Error handling
- Logging
- Configuration

Architecture:

```
Request
↓
Middleware
↓
Route
↓
Controller
↓
Service
↓
Database
↓
Response
```

Build:

REST API #1

Task API

```
POST    /api/tasks
GET     /api/tasks
GET     /api/tasks/:id
PATCH  /api/tasks/:id
DELETE  /api/tasks/:id
```

Add:

```
Validation
Pagination
Filtering
Sorting
Errors
PostgreSQL
```

Test it using:

Postman

or:

Insomnia

or:

Thunder Client

Phase 9 — Authentication & Security

Weeks 19–22

Learn the difference between:

Authentication
Authorization

Then:

Password hashing
Sessions
Cookies
JWT
Access tokens
Refresh tokens
CORS
CSRF
XSS
SQL injection
Rate limiting
Input validation
HTTPS
Environment variables

Important:

Don't just learn:

"How do I implement JWT?"

Understand:

"Why does authentication exist?"

"Where should credentials live?"

"How does the server know who the user is?"

"What happens when the token expires?"

Phase 10 — React

Weeks 21–27+

Now React.

And now React should feel considerably less magical.

Because you already understand:

```
JavaScript
DOM
events
async programming
HTTP
APIs
state
```

React Core

Learn:

```
Components
JSX
Props
State
Events
Conditional rendering
Lists
Keys
Forms
Controlled inputs
```

Then:

```
useState
useEffect
useRef
useContext
```

Then custom hooks.

Understand React's Mental Model

You should be able to explain:

```
State changes
  ↓
React schedules render
  ↓
Component executes
  ↓
New UI description
  ↓
React reconciles
  ↓
DOM updates
```

Don't reduce React to:

"React is HTML inside JavaScript."

It isn't.

React Routing

Learn:

- Routes
- Dynamic routes
- Nested routes
- Navigation
- Protected routes
- Layouts
- 404 routes

React Data

Learn:

- `fetch()`
- loading state
- error state
- success state
- empty state
- pagination
- optimistic updates

Then:

TanStack Query

Only **after** you understand normal `fetch()` .

React State Management

Start:

- Local state

Then:

- Context

Then, only when the application actually requires it:

Zustand

or:

Redux Toolkit

You do **not** need to learn every state management library.

Phase 11 — Full-Stack Integration

Weeks 26–32

Now combine:

```
graph TD
  React --> HTTP
  HTTP --> Express
  Express --> Services
  Services --> PostgreSQL
```

Build:

Full-Stack Task Management System

```
graph TD
  React --> REST_API[REST API]
  REST_API --> Node_js[Node.js]
  Node_js --> Express
  Express --> PostgreSQL
```

Features:

- Registration
- Login
- Logout
- Dashboard
- Create task
- Edit task
- Delete task
- Complete task
- Search
- Filtering
- Pagination
- User profile
- Authorization

This should be a **serious portfolio project**, not a tutorial clone.

Phase 12 — TypeScript

Weeks 30–34

Now introduce:

TypeScript

Learn:

- Primitive types
- Arrays
- Objects
- Interfaces
- Type aliases
- Unions
- Intersections
- Generics
- Narrowing
- Unknown
- Never
- Utility types
- Function types

Then use it with:

- React
- Node
- Express
- API models
- Database layer

The goal:

```
JavaScript knowledge
+
TypeScript
↓
More predictable software
```

Phase 13 — Testing

Weeks 32–36

Testing shouldn't have waited until now—we've been writing testable code throughout—but now we formalize it.

Learn:

```
Unit testing
Integration testing
API testing
Component testing
End-to-end testing
Mocking
Assertions
Test coverage
```

Potential stack:

```
Vitest
React Testing Library
Supertest
Playwright
```

Learn to test:

```
Functions
API endpoints
Components
Authentication
Complete user workflows
```

Phase 14 — Production Engineering

Weeks 35–40+

Now we start moving from:

"I can build an application."

to:

"I can engineer and operate an application."

Learn:

- Docker
- CI/CD
- GitHub Actions
- Environment configuration
- Production builds
- Logging
- Monitoring
- Caching
- Performance
- Security
- DNS
- HTTPS
- Domains
- Reverse proxies

Deploy:

- Frontend
- Backend
- Database

Phase 15 — Next.js

After React is solid.

Not before.

Learn:

- Next.js
- Routing
- Server components
- Client components
- Server-side rendering
- Static generation
- API/server routes
- Caching
- Data fetching

But you should be able to build a React application **without Next.js first**.

Otherwise Next.js becomes a black box.

Phase 16 — Advanced Engineering

This is where you move beyond junior-level fundamentals.

Learn gradually:

Caching
Redis
Queues
Background jobs
WebSockets
Real-time applications
Rate limiting
Advanced SQL
Database optimization
Indexes
Transactions
Distributed systems basics
System design
Observability

Then:

Docker
Kubernetes
Cloud architecture
Infrastructure

Only when there's a reason.

Our Project Ladder

This is one of the biggest improvements I want to make to our roadmap.

Instead of random projects:

```
Project 1
  ↓
Project 2
  ↓
Project 3
  ↓
Project 4
  ↓
Project 5
```

Each project should introduce the **next engineering concept**.

Project 1 — Personal Website

HTML
CSS
Git
GitHub

Project 2 — Calculator

JavaScript
DOM
Events
Functions

Project 3 — Todo Manager

Arrays
Objects
DOM
localStorage
CRUD

Project 4 — Weather Application

Fetch
HTTP
JSON
Promises
async/await
API integration

Project 5 — Vanilla JS Dashboard

Modules
State
DOM architecture
Events
Async operations
Reusable functions

This is our **JavaScript** mastery checkpoint.

Project 6 — Node CLI Application

Node
npm
filesystem
modules
JSON
CLI

Project 7 — REST API

Node
Express
HTTP
REST
PostgreSQL
CRUD
Validation

Project 8 — Authenticated API

Authentication
Authorization
Security
Database
Middleware

Project 9 — React Application

React
Components
Props
State
Hooks
Routing
Forms

Project 10 — Full-Stack Application

React
↓
REST
↓
Node
↓
PostgreSQL

Authentication + CRUD + search + pagination.

Project 11 — Production Application

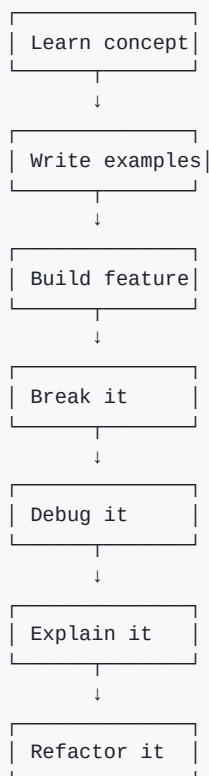
Add:

TypeScript
Testing
Docker
CI/CD
Deployment
Monitoring

This is the project that should eventually sit at the top of your portfolio.

Our Learning Loop

For **every** concept, we'll use this workflow:



That loop is our curriculum.

Our Resource Hierarchy

I strongly agree with the advice about using official documentation.

We'll use this hierarchy:

Tier 1 — Primary

MDN
JavaScript.info
React documentation
Node.js documentation
Express documentation
PostgreSQL documentation
TypeScript documentation

Tier 2 — Structured courses

freeCodeCamp
Full Stack Open
The Odin Project

Tier 3 — YouTube

Useful for:

Visual explanations
Architecture overviews
Project inspiration
Alternative explanations

But YouTube is **not going to become our curriculum.**

Tier 4 — AI

Me.

Use me to:

Explain
Quiz you
Review code
Debug with you
Generate exercises
Challenge your understanding
Explain documentation
Design projects
Review architecture

But I'll also tell you when I think you're trying to skip a concept.



Our Milestone System

Instead of saying:

"I studied JavaScript for two months."

we'll say:

Level 1 — Syntax

Can you write JavaScript?



Level 2 — Problem Solving

Can you solve problems?



Level 3 — Browser

Can you build interactive applications?



Level 4 — Async

Can you work with APIs and asynchronous code?



Level 5 — Backend

Can you build an HTTP server?



Level 6 — Database

Can you persist and query data?



Level 7 — API Engineering

Can you design a REST API?



Level 8 — React

Can you build complex interfaces?



Level 9 — Full Stack

Can you connect everything?



Level 10 — Production

Can you test, deploy and maintain it?



The Roadmap At A Glance

Phase	Approx. time	Main focus	Project
0	Days 1–3	Setup/Git/Linux	Hello World
1	Weeks 1–3	HTML/CSS	Portfolio
2	Weeks 3–8	JS fundamentals	Todo
3	Weeks 6–9	Runtime/async	JS exercises
4	Weeks 7–10	DOM/browser	Dashboard
5	Weeks 9–11	HTTP/REST	API client
6	Weeks 11–14	Node	CLI
7	Weeks 13–17	PostgreSQL/SQL	Database
8	Weeks 16–20	Express/REST	REST API
9	Weeks 19–22	Auth/security	Auth API
10	Weeks 21–27	React	SPA
11	Weeks 26–32	Full stack	Full-stack app
12	Weeks 30–34	TypeScript	TS migration
13	Weeks 32–36	Testing	Tested app
14	Weeks 35–40+	Deployment/DevOps	Production
15	Later	Next.js	Modern full-stack
16	Advanced	Systems	Production-scale systems

Don't interpret those weeks as deadlines.

They're **rough progression markers**.

If JavaScript takes you 12 weeks instead of 8, that's fine.

I'd rather have you spend 12 weeks understanding JavaScript properly than rush into React and spend the next year confused about why things work.

And Here's How I Suggest We Actually Work Together

This is the part I would add to the roadmaps you received.

Instead of you disappearing for six months and returning saying:

"I've finished JavaScript."

we can run this as an **ongoing developer apprenticeship**.

For each phase, I can act as your:

teacher → **pair programmer** → **reviewer** → **interviewer** → **debugger** → **mentor**.

For example:

When we're learning functions

I'll give you:

```
Concept
↓
Explanation
↓
Small exercise
↓
Challenge
↓
Code review
↓
Real-world exercise
```

Then I won't let you move forward simply because you can repeat the syntax.

I'll check whether you actually understand it.

Our First Milestone

We're **not** starting with React.

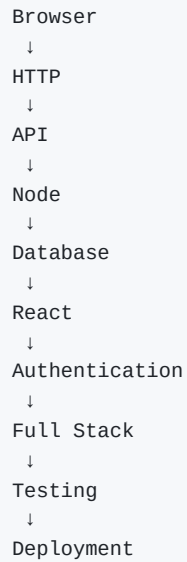
We're starting here:

```
PHASE 0
↓
Developer environment
↓
Git
↓
Terminal
↓
Node.js
↓
JavaScript
```

Your first actual milestone will be:

Build and explain a small vanilla JavaScript application without following a tutorial.

Once you can do that, we'll progressively add:



```
graph TD; Browser --> HTTP; HTTP --> API; API --> Node; Node --> Database; Database --> React; React --> Authentication; Authentication --> Full_Stack[Full Stack]; Full_Stack --> Testing; Testing --> Deployment;
```

Browser
↓
HTTP
↓
API
↓
Node
↓
Database
↓
React
↓
Authentication
↓
Full Stack
↓
Testing
↓
Deployment

And by the time we reach React, you won't be someone who merely **knows React syntax**.

You'll understand the machinery underneath it.

That's the version of this roadmap I would personally recommend for you: **take the practical structure from the first roadmap, the runtime/engineering depth from the senior developer's roadmap, and combine them into a slower apprenticeship-style progression where every phase is proven through projects.**